NTUEE DCLab

LAB 3: Digital Voice Recorder

Lecturer: Hao-Qun Lin, Yu-Chuan Chen

Dept. of Electrical Engineering, NTU



國立臺灣大學電機工程學系

數位電路實驗室

AMD



Outline

- Introduction
  - Lab requirements
- Memory devices
- Implementation
  - System architecture
  - Digital signal processing (DSP)
  - Clock
- Code template
- Report regulations



Introduction

- Digital voice recorder.
  - Use the FPGA's onboard digital microphone and the computer speakers to record and play back PDM audio signals.
  - Perform signal processing on the audio data.
  - Access the memory module to store and retrieve audio data.

Lab Requirements

- The system should support the following features:
  - Record and playback.
  - Pause and stop during playback.
  - Support a 3.125 MHz clock frequency and provide up to 16 seconds of recording time.
  - Support fast playback (2×, 3×, 4×, 5×, 6×, 7×, and 8×) and slow playback (1/2×, 1/3×, 1/4×, 1/5×, 1/6×, 1/7×, and 1/8×).

- Bonus (Must be clearly demonstrated and explained in the demo.)
  - Display the operating status of the voice recorder using additional hardware modules.
  - Implement additional audio signal processing features or other creative enhancements.



Outline

- Introduction
  - Lab requirements
- Memory devices
- Implementation
  - System architecture
  - Digital signal processing (DSP)
  - Clock
- Code template
- Report regulations

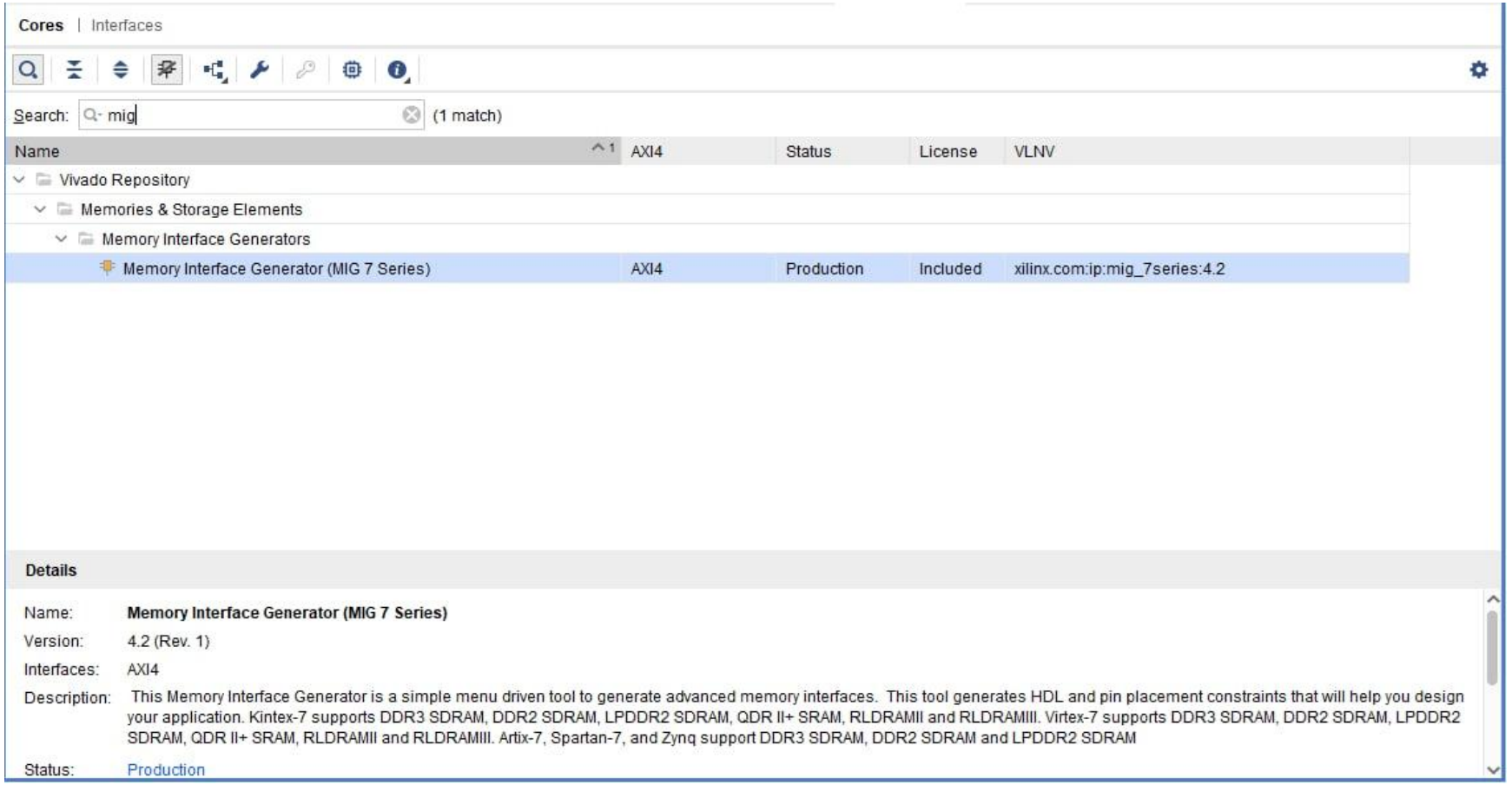


Memory Devices : (128MiB) DDR2 SDRAM

- Access the user interface signal
- Instantiate the provided ddr2_interface module to read and write to DDR2 in a simple way.

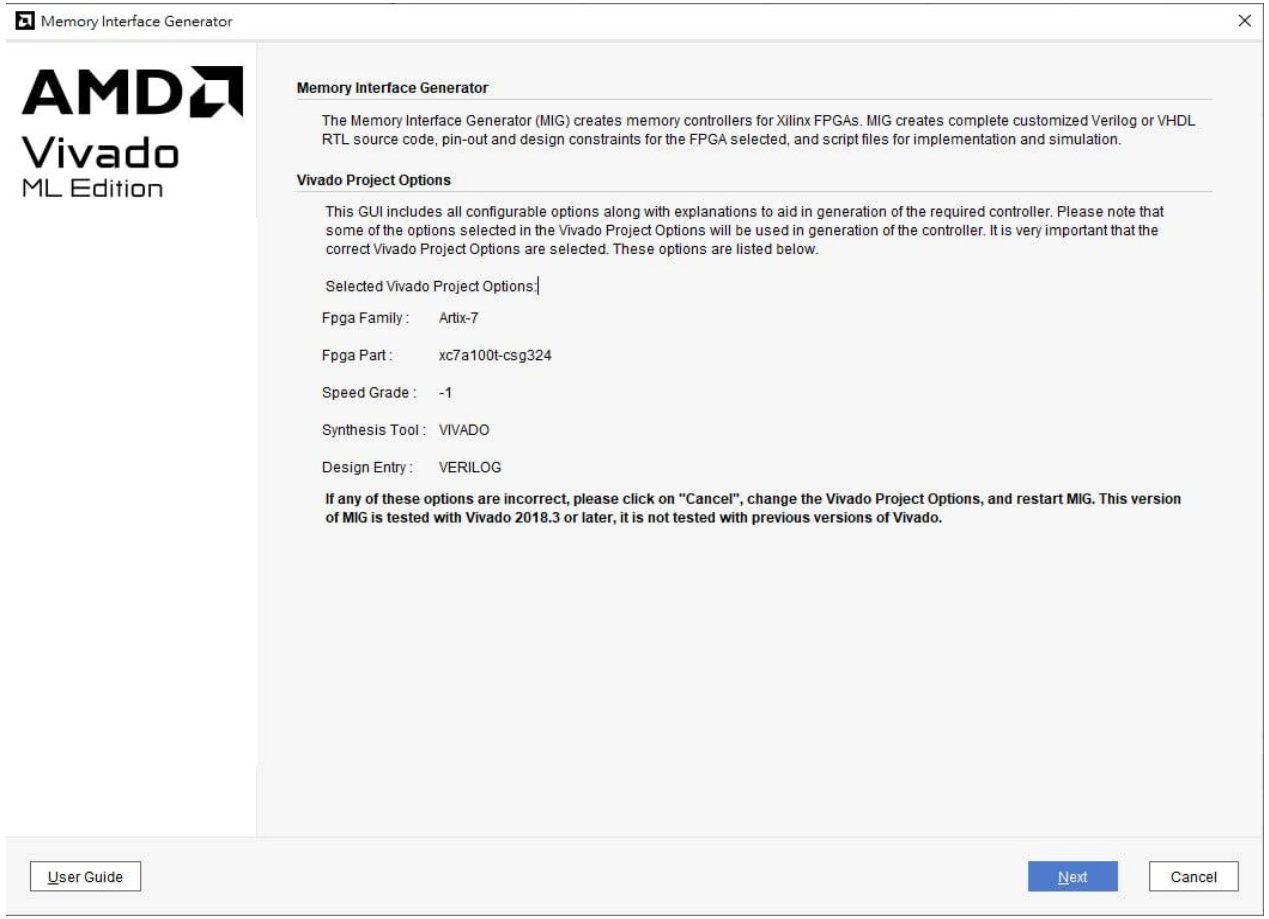
Memory Devices : (128MiB) DDR2 SDRAM

- IP catalog => Memory Interface Generator (MIG 7 Series)



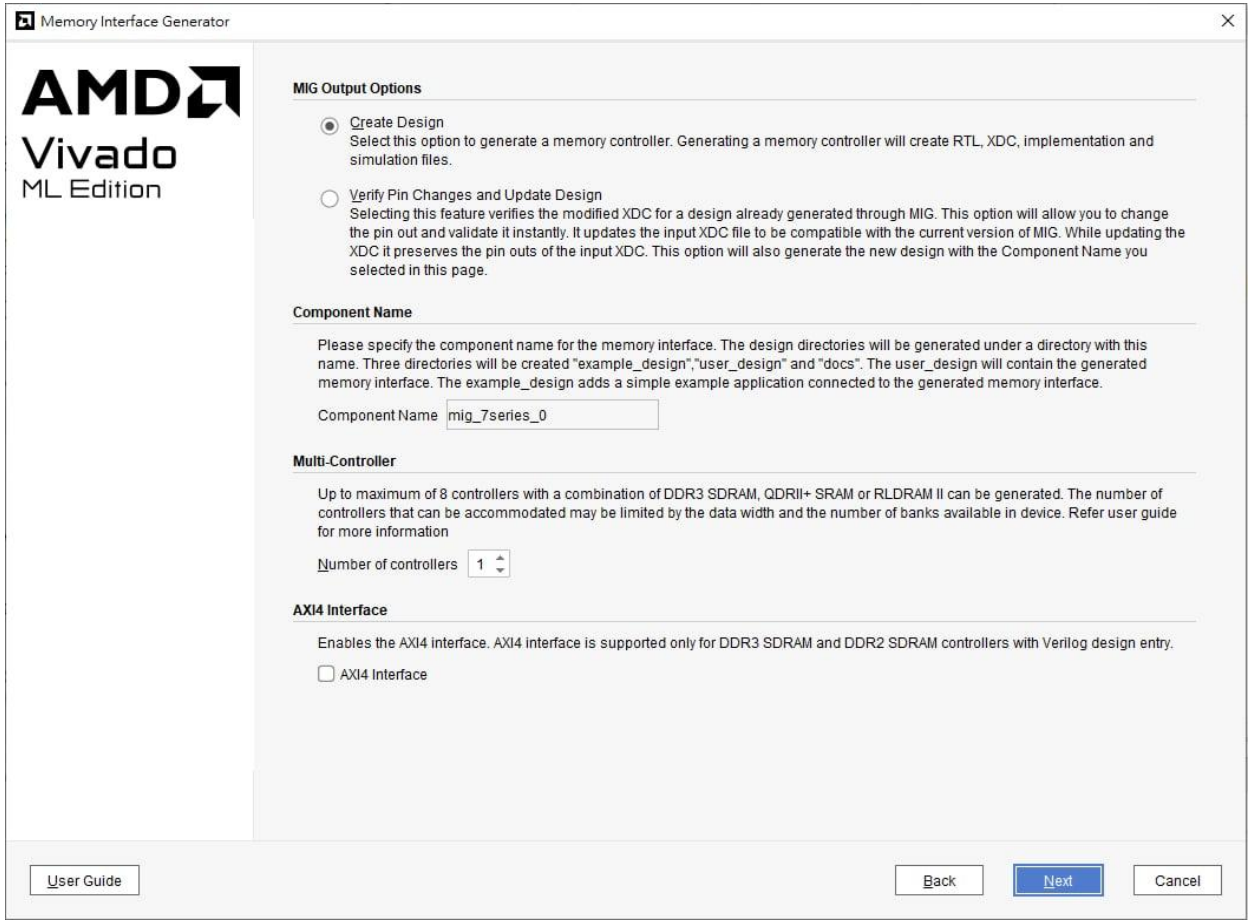
Memory Devices : (128MiB) DDR2 SDRAM

- Make sure FPGA Part: xc7a100t-csg324 => Next



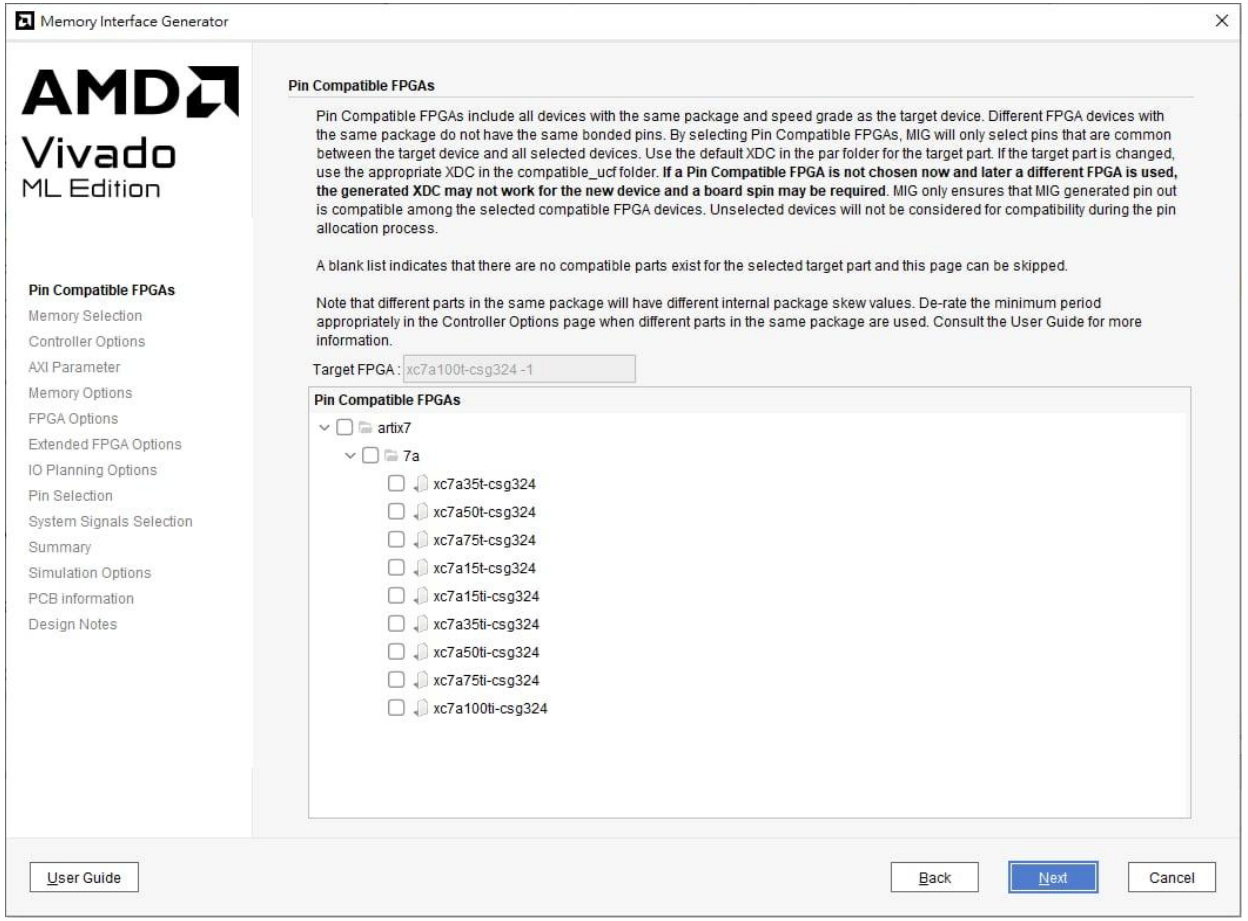
Memory Devices : (128MiB) DDR2 SDRAM

- Set Component Name: mig_7series_0 => Next



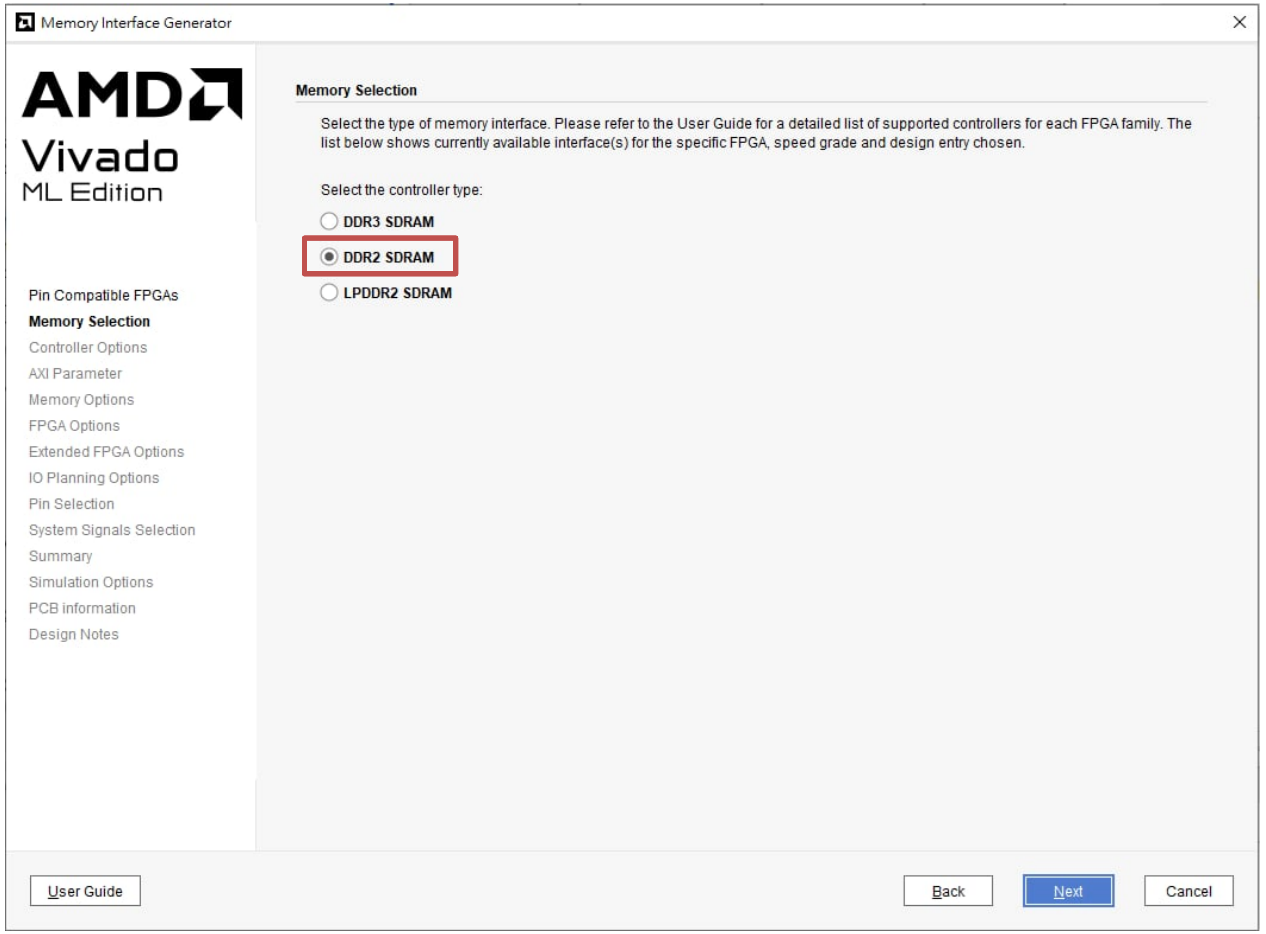
Memory Devices : (128MiB) DDR2 SDRAM

- Next



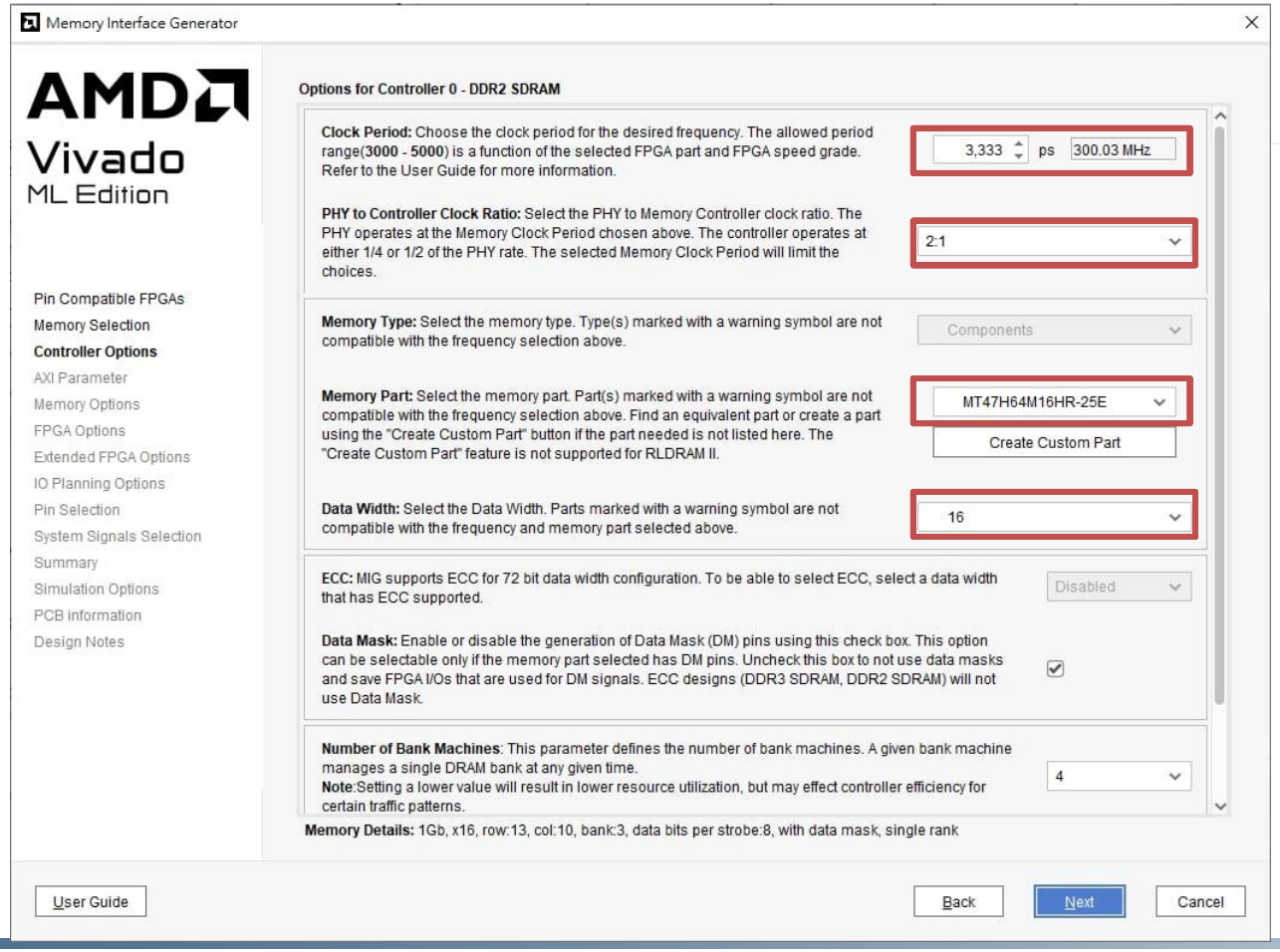
Memory Devices : (128MiB) DDR2 SDRAM

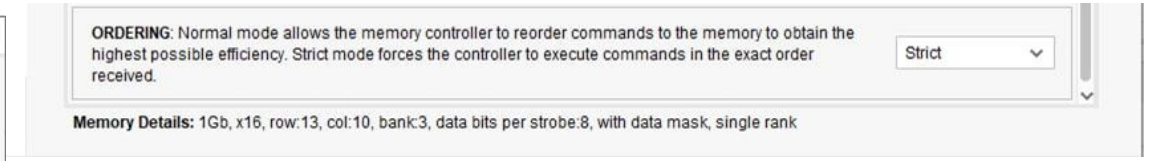
- Select DDR2 SDRAM => Next



Memory Devices : (128MiB) DDR2 SDRAM

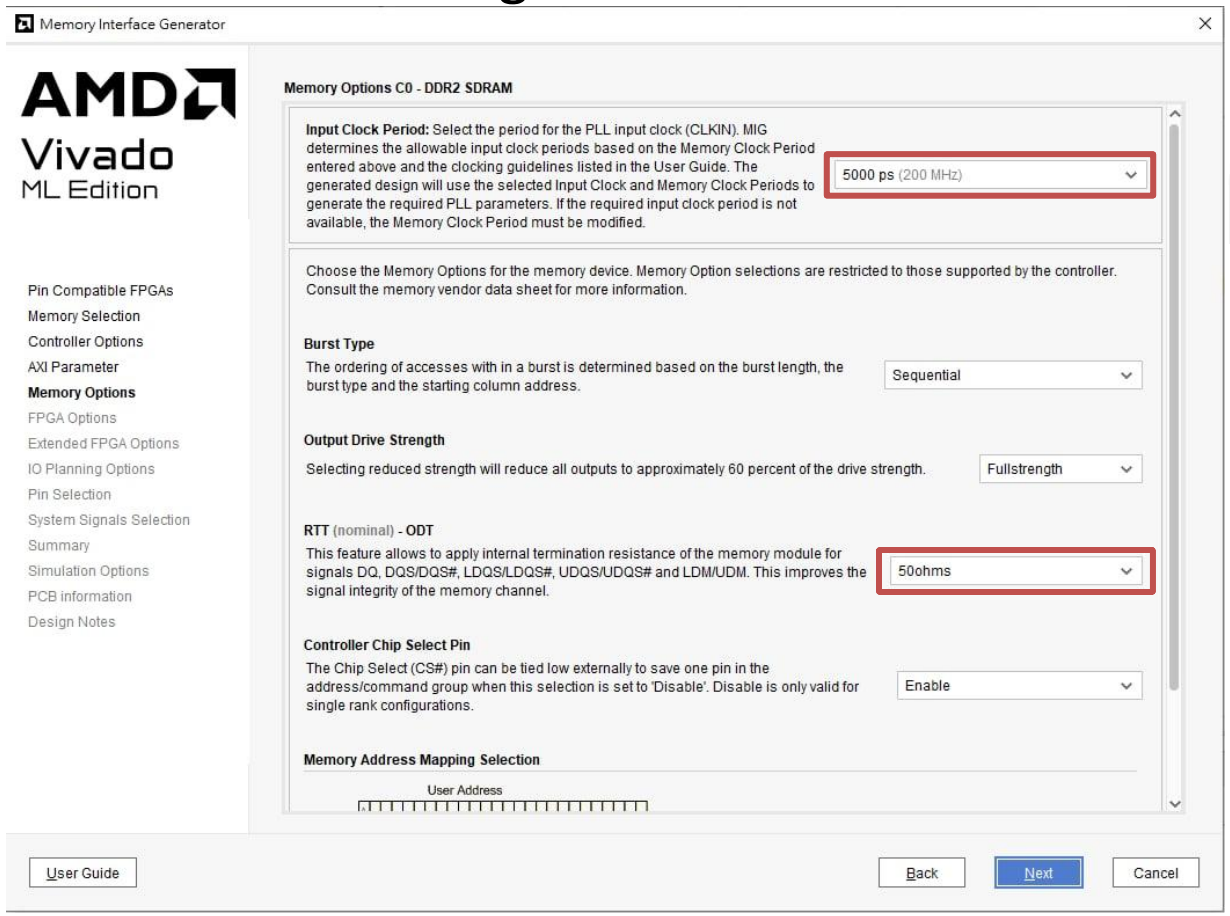
- Set as following screenshot => Next





Memory Devices : (128MiB) DDR2 SDRAM

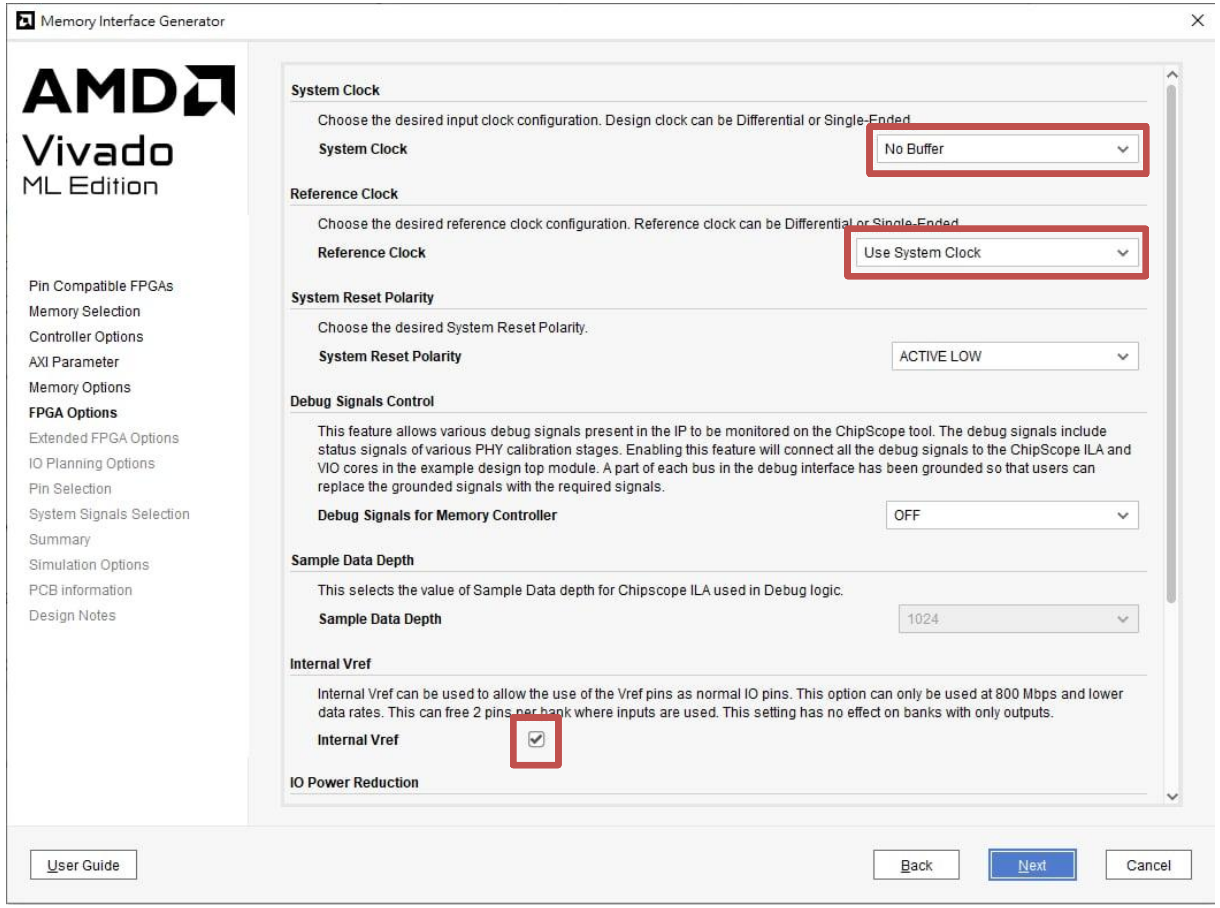
- Set as following screenshot => Next

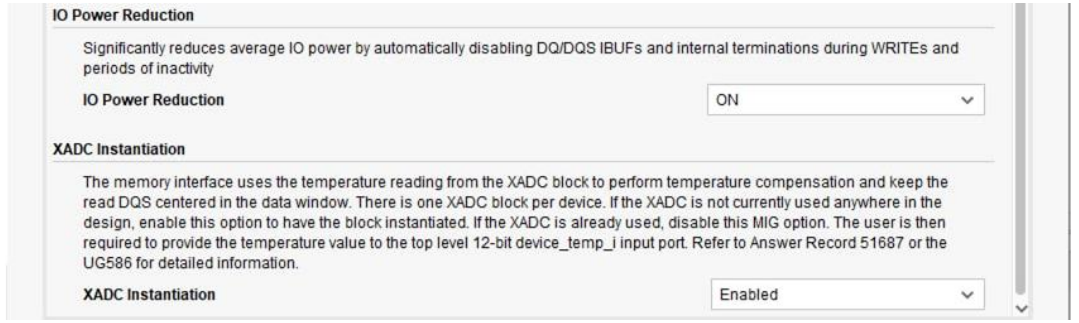




Memory Devices : (128MiB) DDR2 SDRAM

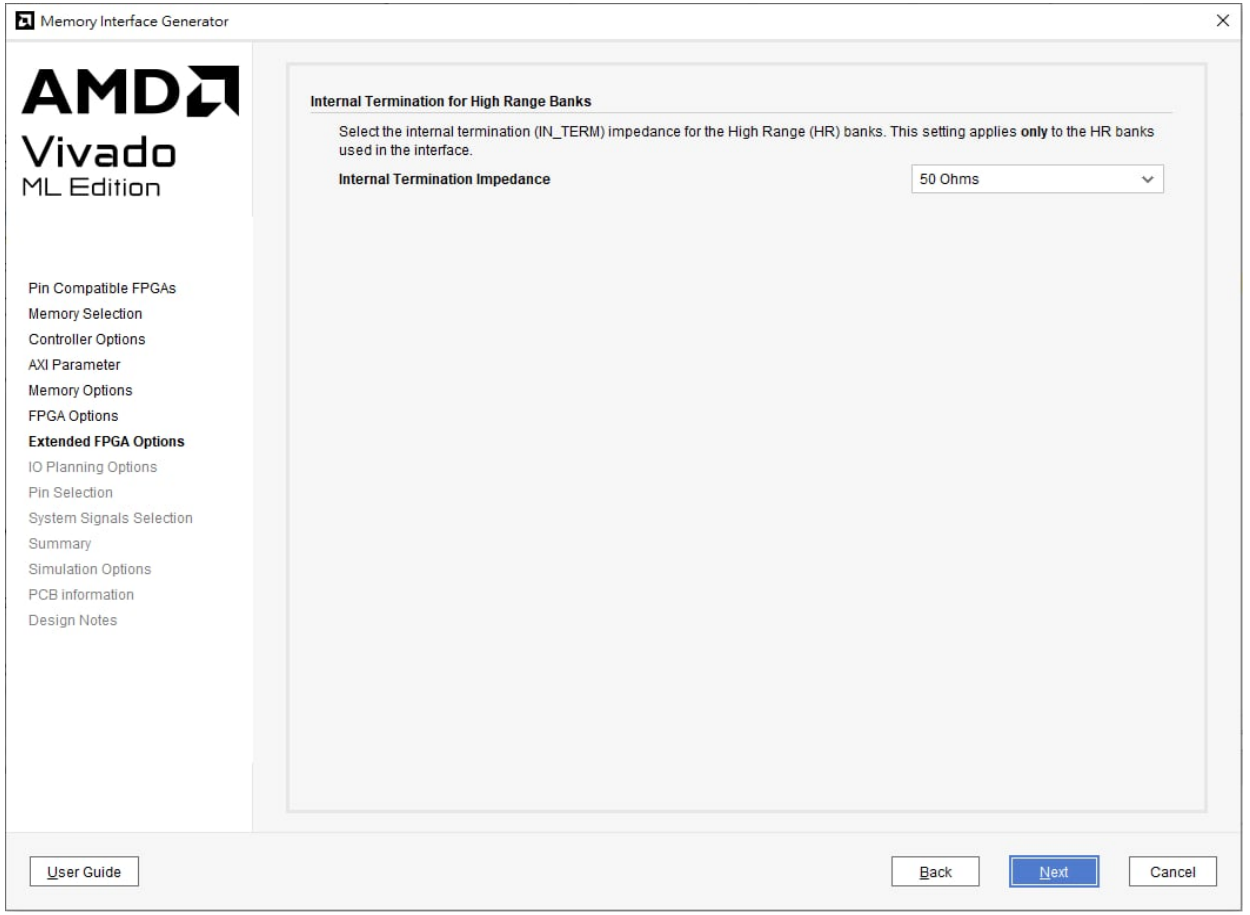
- Set as following screenshot => Next





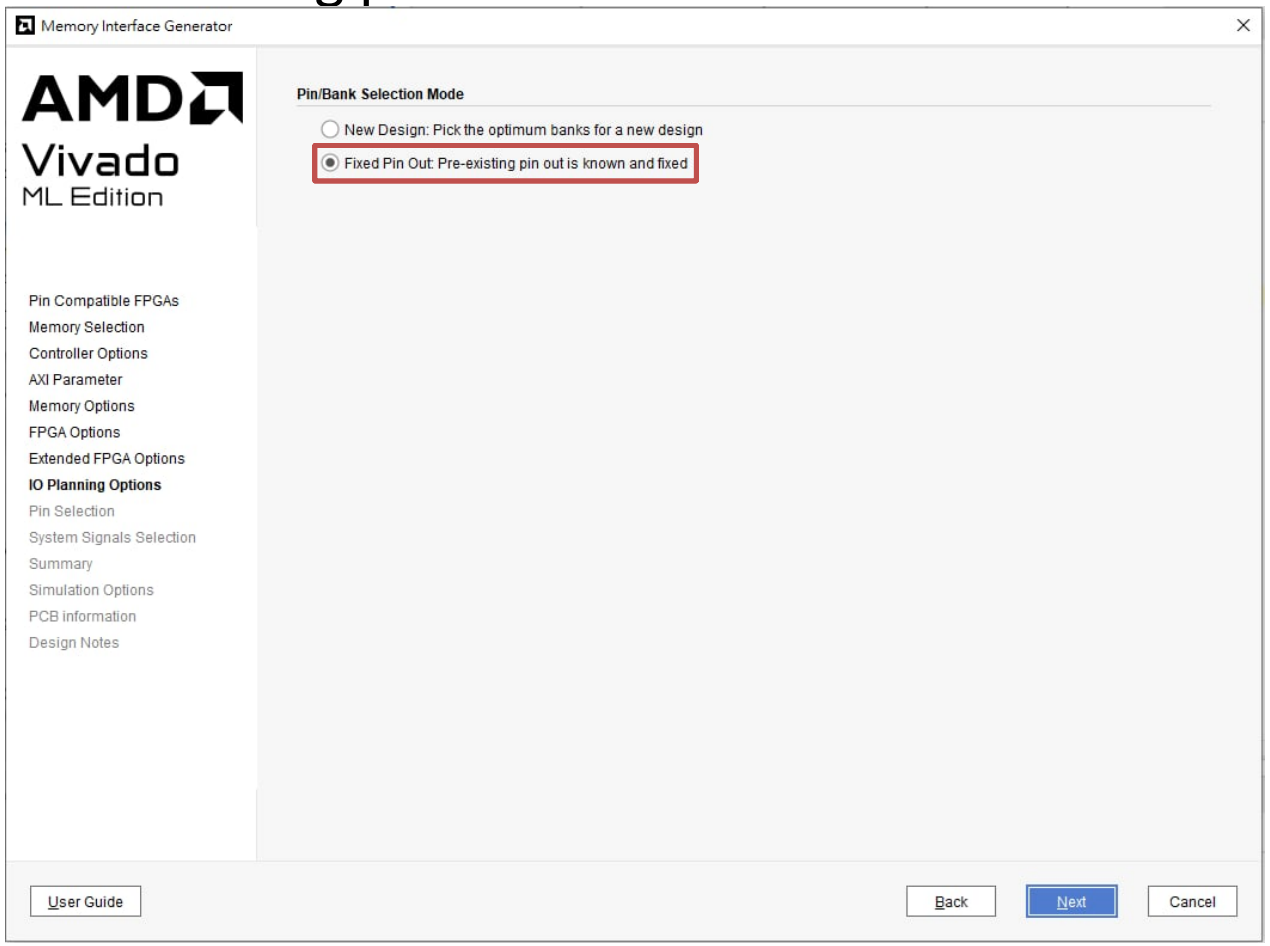
Memory Devices : (128MiB) DDR2 SDRAM

- Set as following screenshot => Next



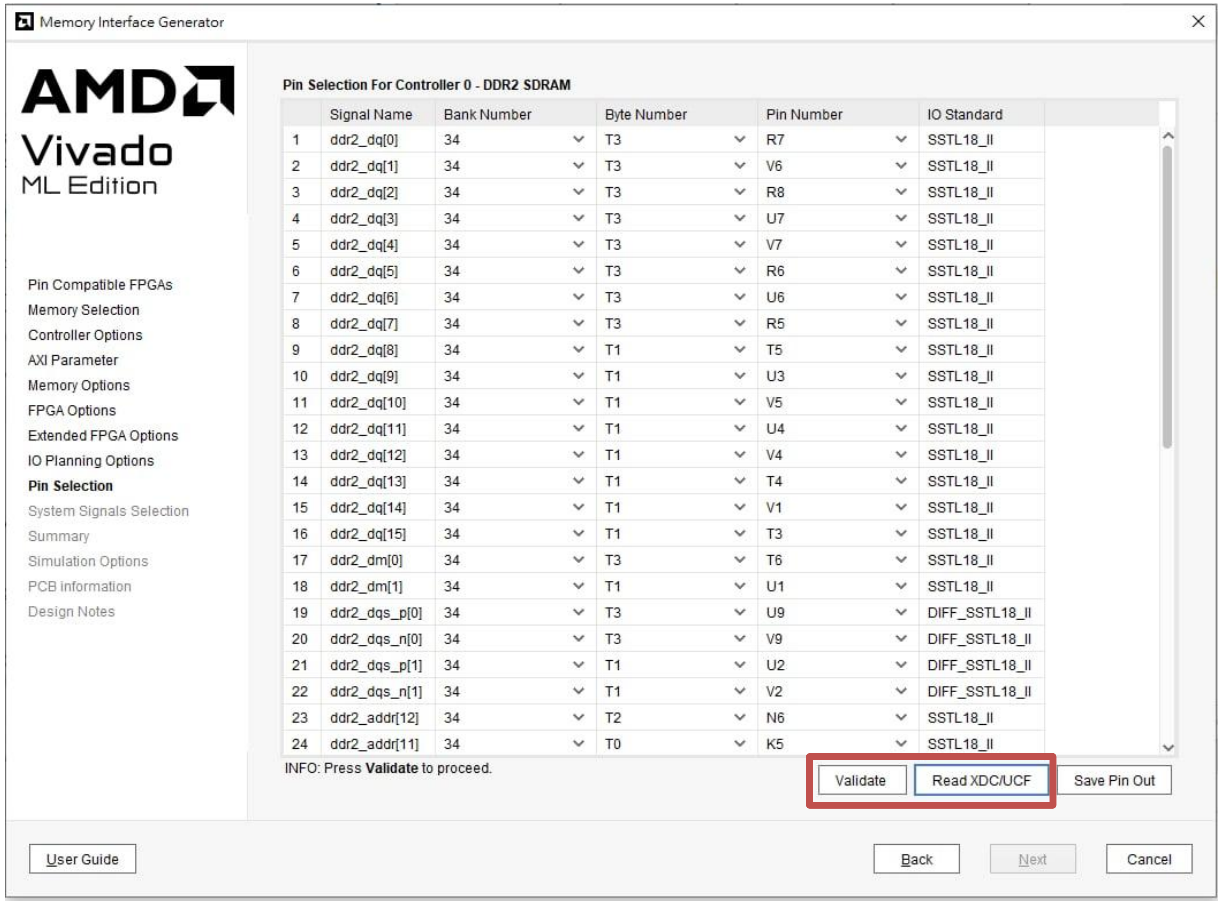
Memory Devices : (128MiB) DDR2 SDRAM

- Fixed Pin Out: Pre-existing pin out is known and fixed => Next



Memory Devices : (128MiB) DDR2 SDRAM

- Read XDC/UCF => Select the given mig.ucf => Validate => Next

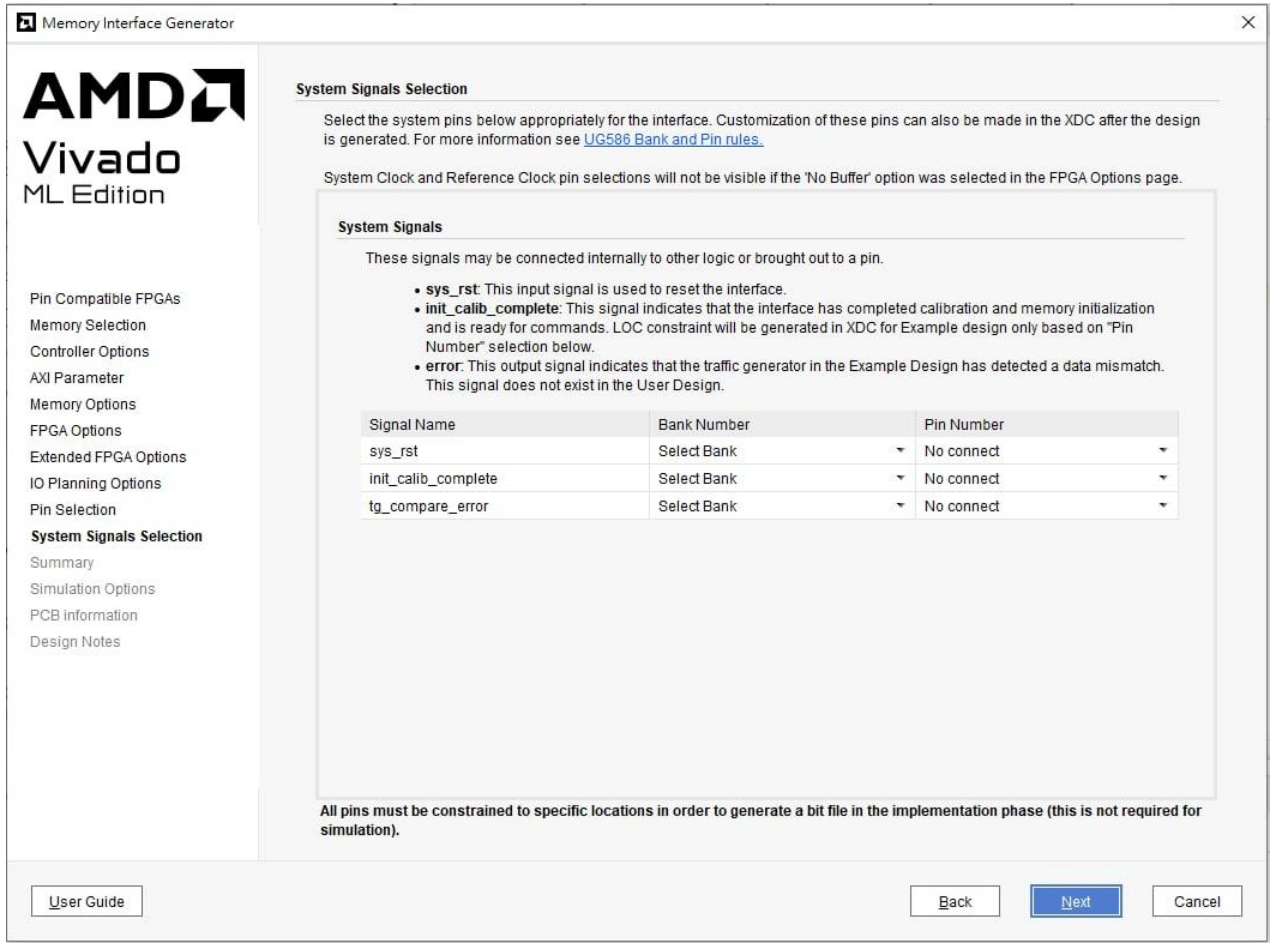


	Signal Name	Bank Number	Byte Number	Pin Number	IO Standard
1	ddr2_dq[0]	34	T3	R7	SSTL18_II
2	ddr2_dq[1]	34	T3	V6	SSTL18_II
3	ddr2_dq[2]	34	T3	R8	SSTL18_II
4	ddr2_dq[3]	34	T3	U7	SSTL18_II
5	ddr2_dq[4]	34	T3	V7	SSTL18_II
6	ddr2_dq[5]	34	T3	R6	SSTL18_II
7	ddr2_dq[6]	34	T3	U6	SSTL18_II
8	ddr2_dq[7]	34	T3	R5	SSTL18_II
9	ddr2_dq[8]	34	T1	T5	SSTL18_II
10	ddr2_dq[9]	34	T1	U3	SSTL18_II
11	ddr2_dq[10]	34	T1	V5	SSTL18_II
12	ddr2_dq[11]	34	T1	U4	SSTL18_II
13	ddr2_dq[12]	34	T1	V4	SSTL18_II
14	ddr2_dq[13]	34	T1	T4	SSTL18_II
15	ddr2_dq[14]	34	T1	V1	SSTL18_II
16	ddr2_dq[15]	34	T1	T3	SSTL18_II
17	ddr2_dm[0]	34	T3	T6	SSTL18_II
18	ddr2_dm[1]	34	T1	U1	SSTL18_II
19	ddr2_dqs_p[0]	34	T3	U9	DIFF_SSTL18_II
20	ddr2_dqs_n[0]	34	T3	V9	DIFF_SSTL18_II
21	ddr2_dqs_p[1]	34	T1	U2	DIFF_SSTL18_II
22	ddr2_dqs_n[1]	34	T1	V2	DIFF_SSTL18_II
23	ddr2_addr[12]	34	T2	N6	SSTL18_II
24	ddr2_addr[11]	34	T0	K5	SSTL18_II



Memory Devices : (128MiB) DDR2 SDRAM

- Next



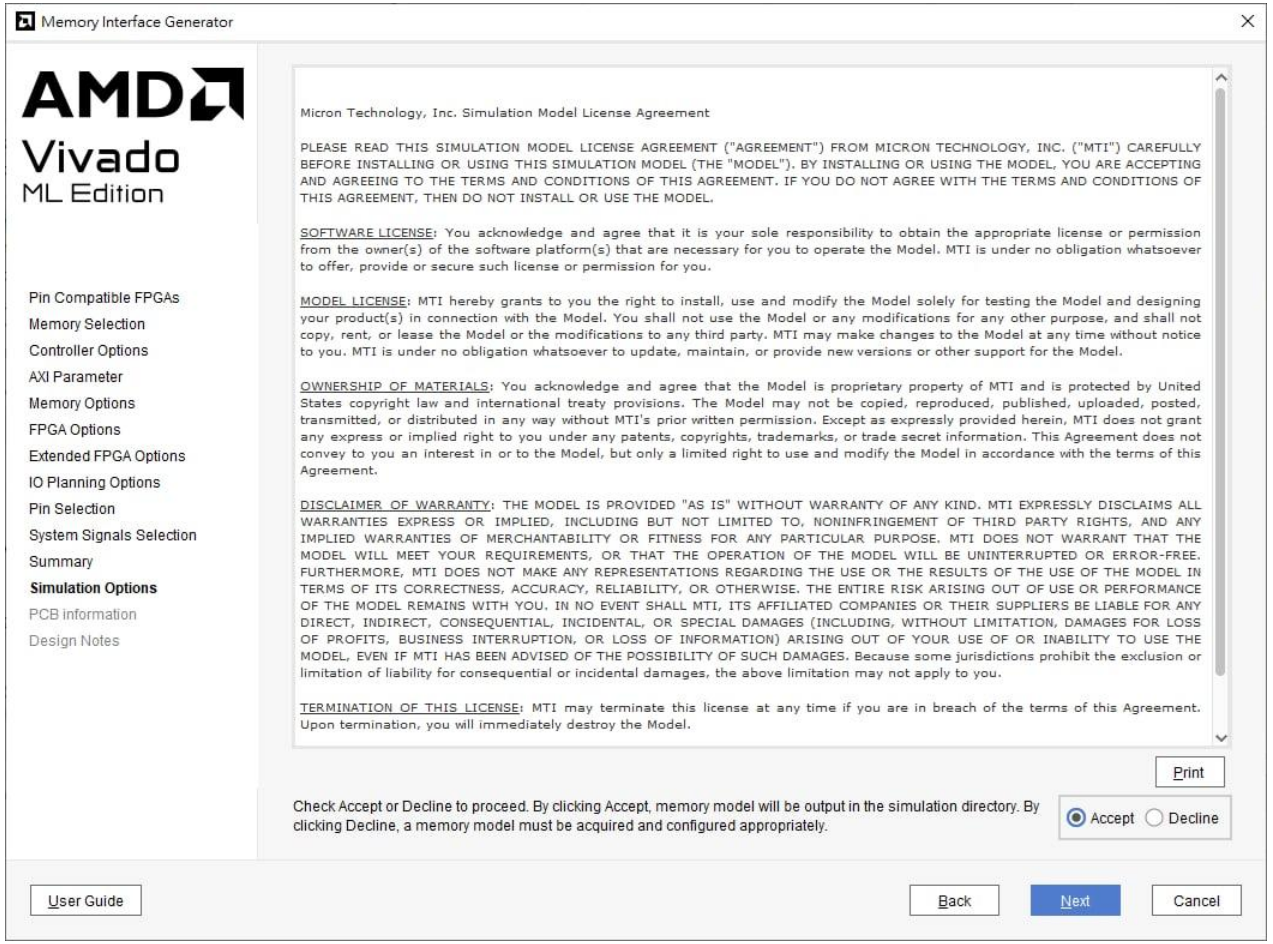
Memory Devices : (128MiB) DDR2 SDRAM

- Next



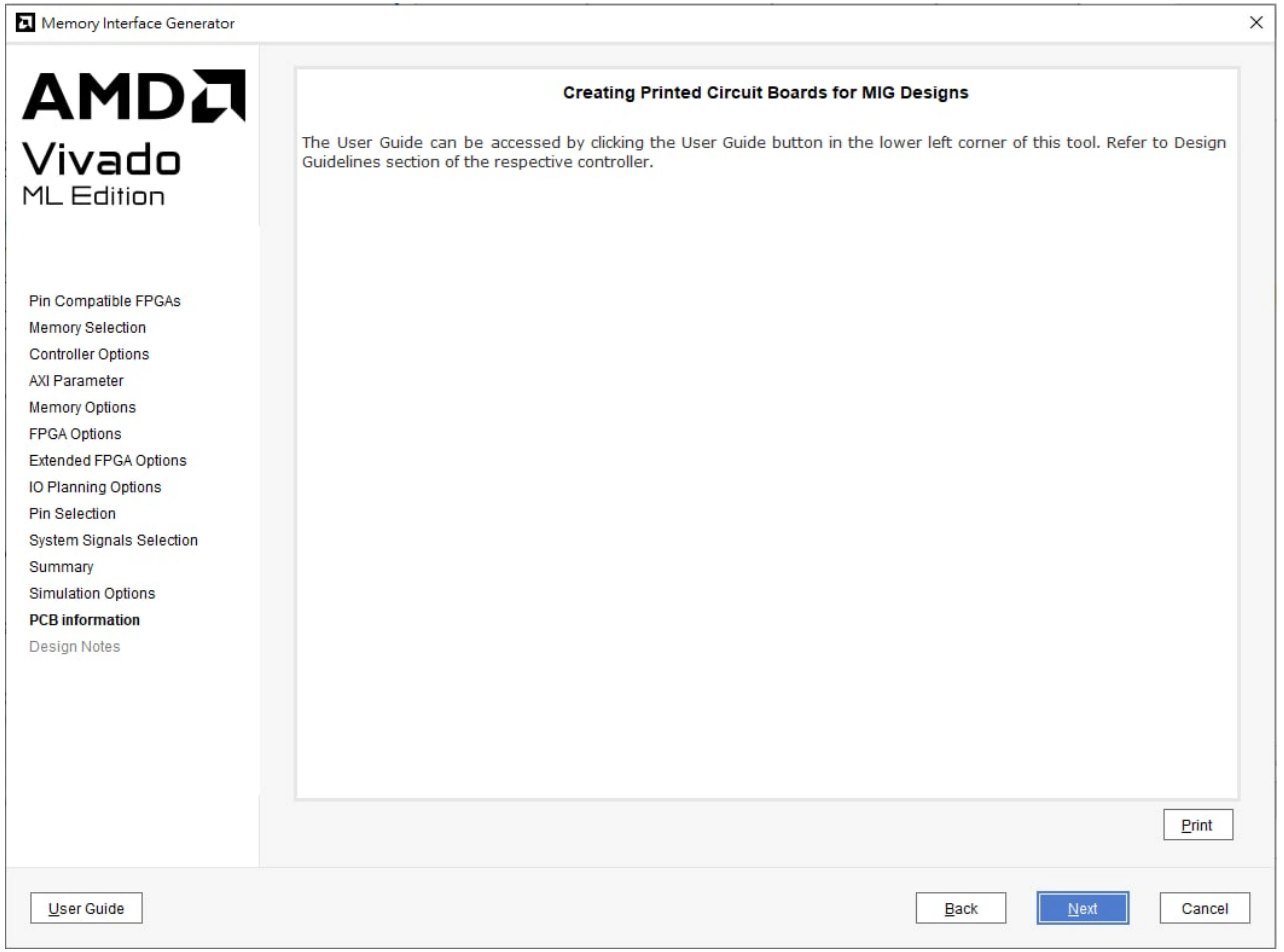
Memory Devices : (128MiB) DDR2 SDRAM

- Accept => Next



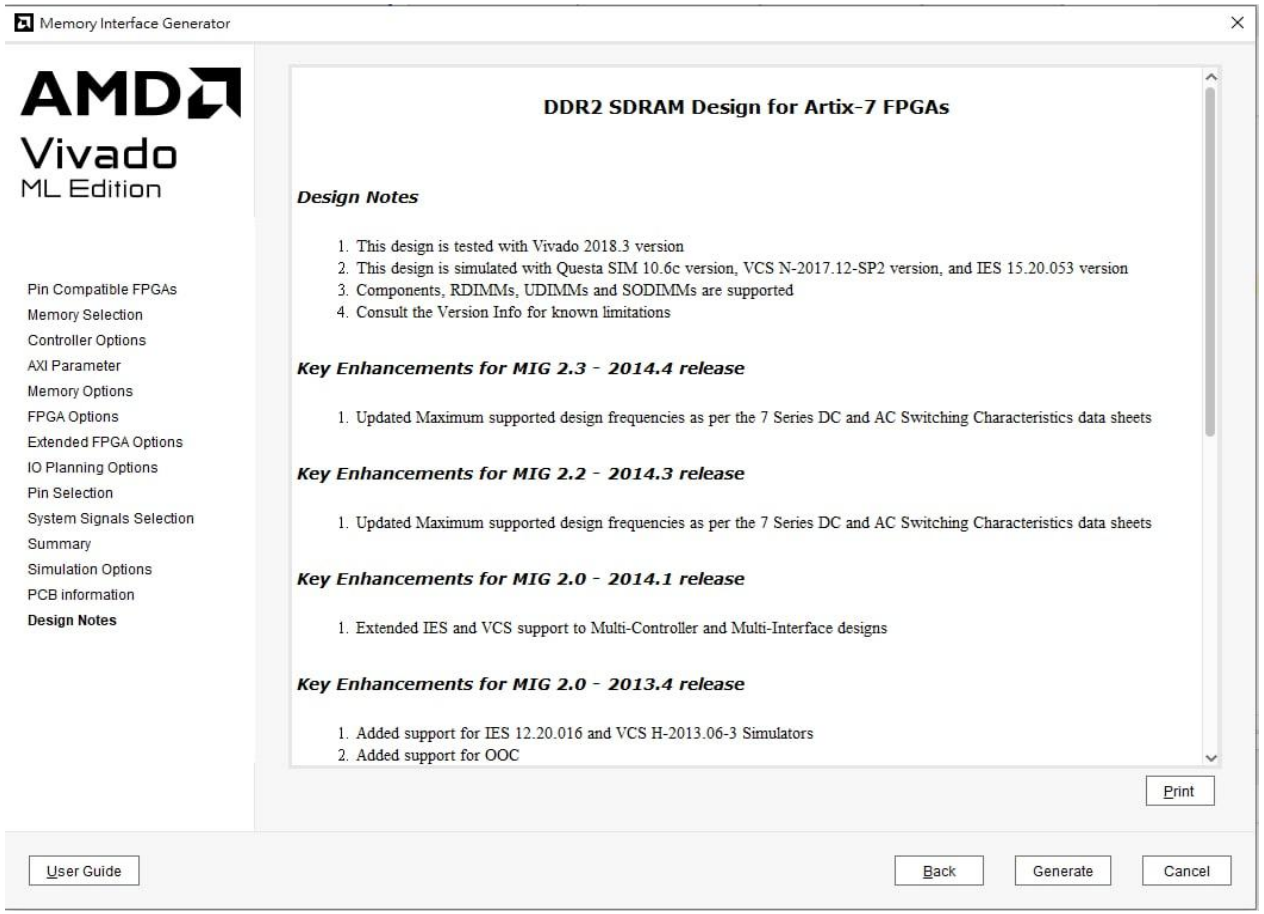
Memory Devices : (128MiB) DDR2 SDRAM

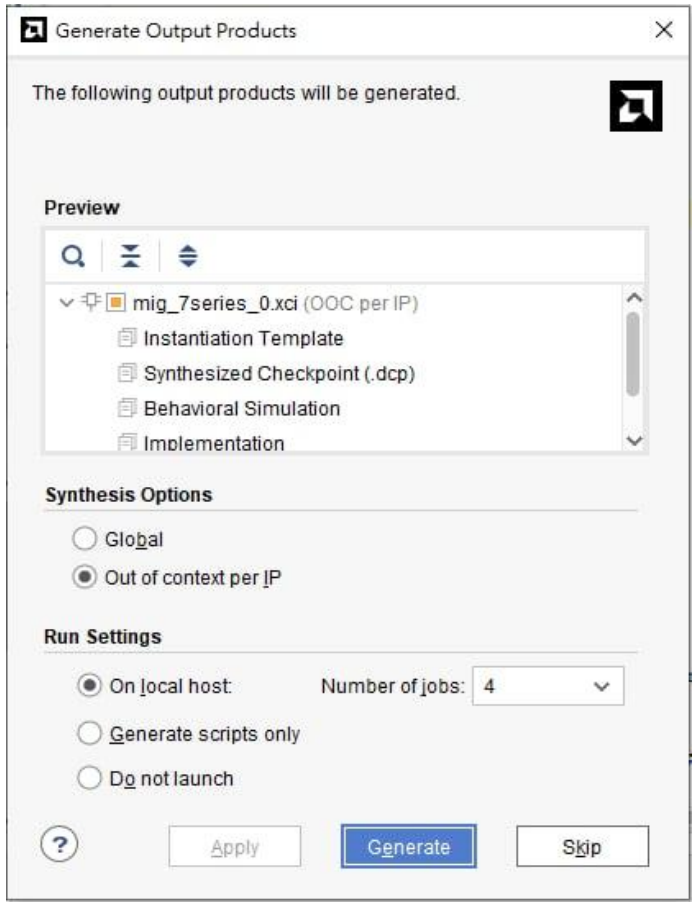
- Next



Memory Devices : (128MiB) DDR2 SDRAM

- Generate => Generate







Memory Devices : (128MiB) DDR2 SDRAM

- mig_7series_0 Interface

Signal Name	I/O	Width	Description
ddr2_...	X	X	Physical signal interface of ddr2
app_addr	Input	27	Byte address for ddr2
app_cmd	Input	3	Read/Write Command
app_en	Input	1	Enable signal
app_wdf_data	Input	64	Write data
app_wdf_end	Input	1	End of writing signal
app_wdf_wren	Input	1	Write enable
app_rd_data	Output	64	Read_data
app_rd_data_end	Output	1	End of reading signal
app_rd_data_valid	Output	1	Valid reading data
app_rdy	Output	1	Ready signal



Memory Devices : (128MiB) DDR2 SDRAM

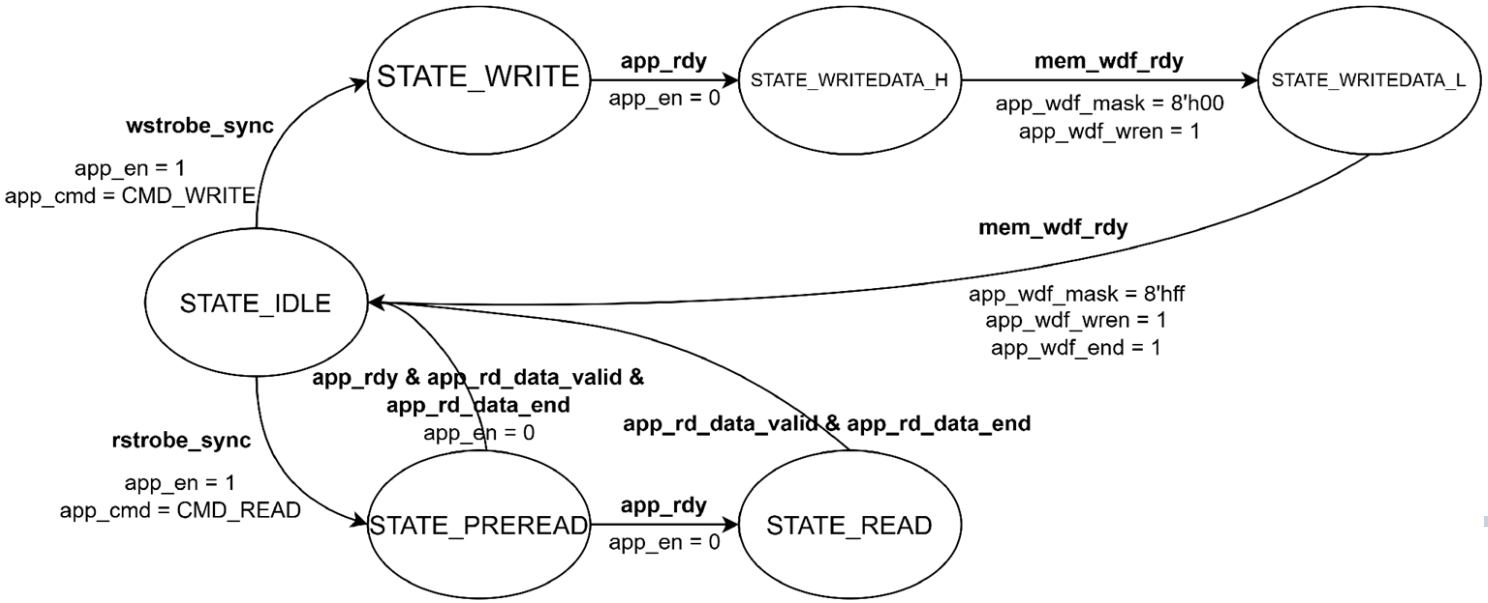
- mig_7series_0 Interface

Signal Name	I/O	Width	Description
app_wdf_rdy	Output	1	Write ready signal
ui_clk	Output	1	User interface clock
ui_clk_sync_rst	Output	1	Synchrounus reset for user interface clock
app_wdf_mask	Input	8	Writing byte mask
sys_clk_i	Input	1	System clock
sys_rst	Input	1	System reset

- ff_sync, flag_sync: Clock Domain Crossing Module

Memory Devices : (128MiB) DDR2 SDRAM

- This lab primarily uses DDR2 SDRAM.
  - With a sampling frequency of 3.125 MHz, the memory can store up to 343.6 seconds of audio.
- Example FSM for read and write using ddr2_interface
  - Read data could be available at both PREREAD and READ state
  - Write the data at WRITEDATA_H state
  - Mask the writing port in WRITEDATA_L state





Memory Devices : (128MiB) DDR2 SDRAM

- ddr2_interface Interface

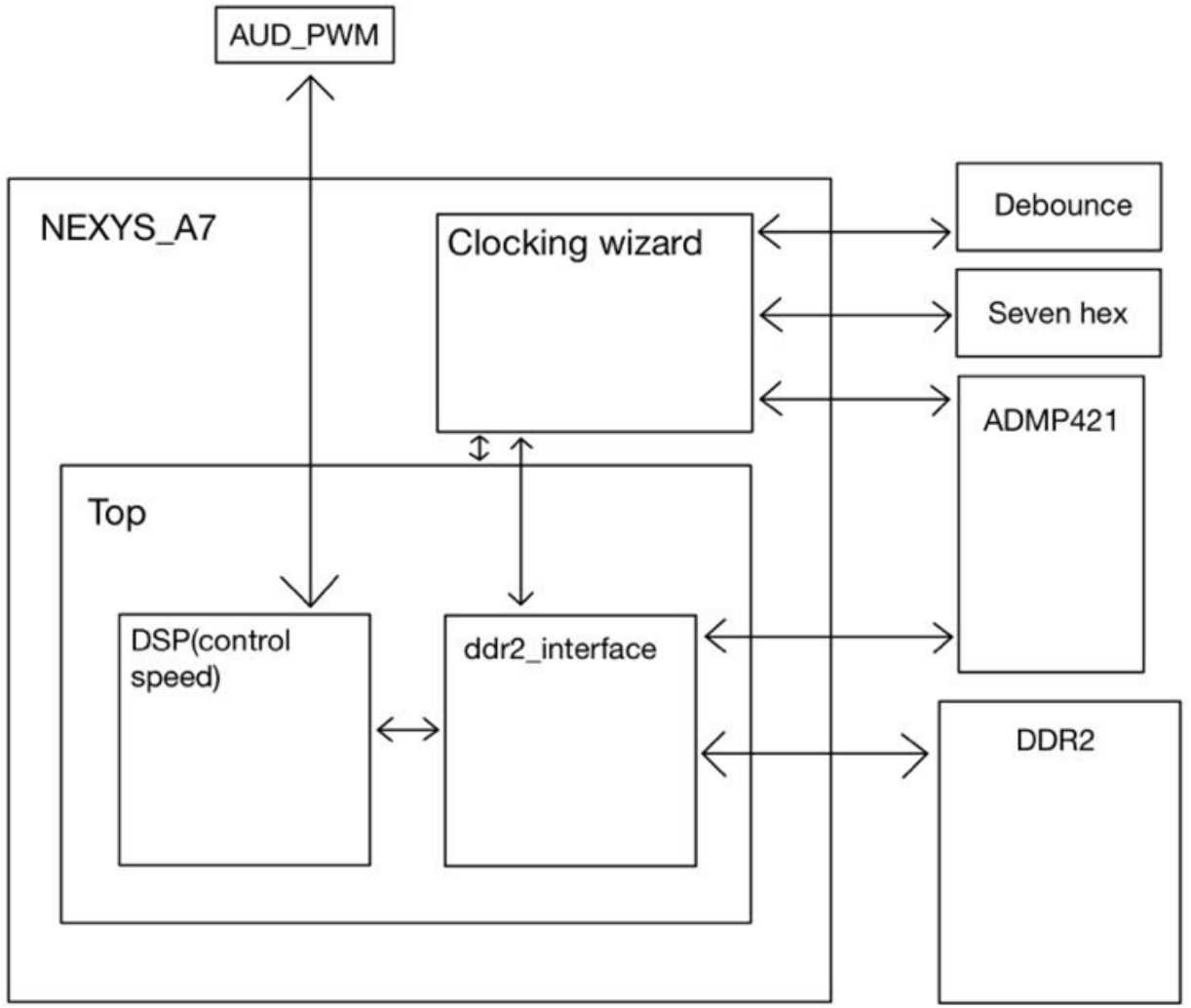
Signal Name	I/O	Width	Description
ddr2_...	X	X	Physical signal interface of ddr2
clk_mem	Input	1	200MHz Clock for DDR2
rst_n	Input	1	Active low reset signal for DDR2
cpu_clk	Input	1	Connected to 3.125MHz clk in Top.v
addr	Input	24	Write and read address
data_in	Input	64	Data to be write to DDR2
data_out	Output	64	Data read from DDR2
rstrobe	Input	1	rstrobe = 1 when reading and 0 otherwise
wstrobe	Input	1	wstrobe = 1 when writing and 0 otherwise
transaction_complete	Output	1	Finish reading or writing
ready	Output	1	Ready to read or write



Outline

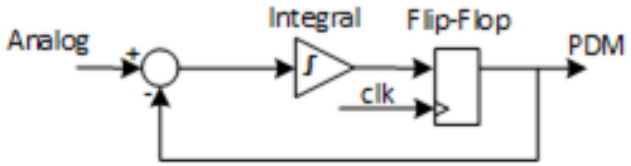
- Introduction
  - Lab requirements
- Memory devices
- Implementation
  - System architecture
  - Digital signal processing (DSP)
  - Clock
- Code template
- Report regulations

System Architecture



ADMP421 Microphone: PDM Signal

- The Nexys A7 board includes an microphone using an Analog Device ADMP421. The digitized audio is output in the pulse density modulated (PDM) format.



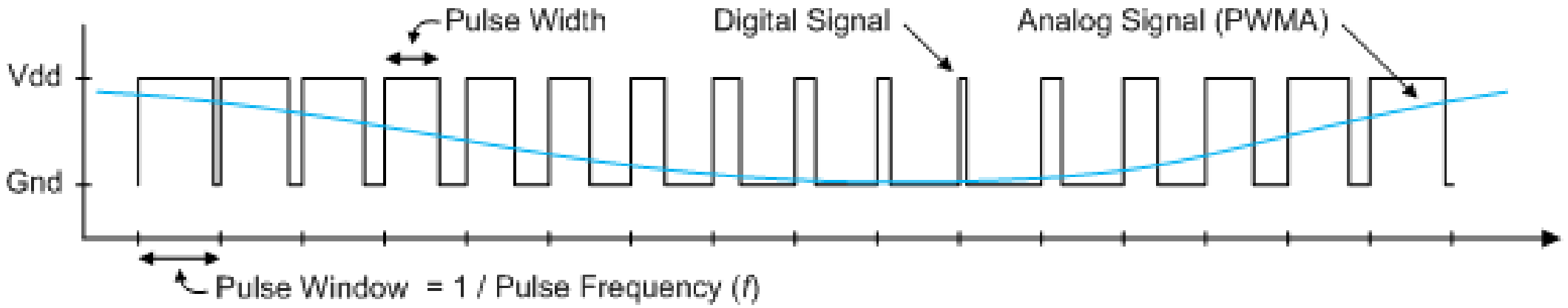
- A PDM signal is generated from an analog signal through a process called delta-sigma modulation. The input of the flip-flop acts like a comparator (any signal above Vdd/2 is considered as '1' and any input bellow Vdd/2 is considered '0'). The input of the integral circuit is the difference of the input analog signal and the PDM signal of the previous clock cycle.

Table 14.1.2. Sigma Delta Modulator with a 0.4Vdd input.

Sum	Integrator Out	Flip-flop Output
0.4-0=0.4	0+0.4=0.4	0
0.4-0=0.4	0.4+0.4=0.8	1
0.4-1=-0.6	0.8-0.6=0.2	0
0.4-0=0.4	0.2+0.4=0.6	1
0.4-1=-0.6	0.6-0.6=0	0
0.4-0=0.4	0+0.4=0.4	0
0.4-0=0.4	0.4+0.4=0.8	1
0.4-1=-0.6	0.8-0.6=0.2	0

Mono Audio Output(PWM or PDM Signal)

- The on-board audio jack is driven by a Low-pass 4th Order Filter that provides mono audio output. The input of the filter is AUD_PWM. A digital input will typically be a pulse-width modulated (PWM) or pulse density modulated (PDM) open-drain signal produced by the FPGA.



Digital Signal Processing (DSP)

- Fast Playback
  - Downsampling
  - To achieve an n× playback speed, reduce both the total number of samples and the number of logic-1 samples to 1/n of the original.
  - Use an accumulator to determine the output. Output one logic-1 sample whenever n logic-1 samples have been accumulated; otherwise, output a logic-0 sample.
- Slow Playback
  - Upsampling
  - Repeat each sample n times to achieve a playback speed of 1/n×.

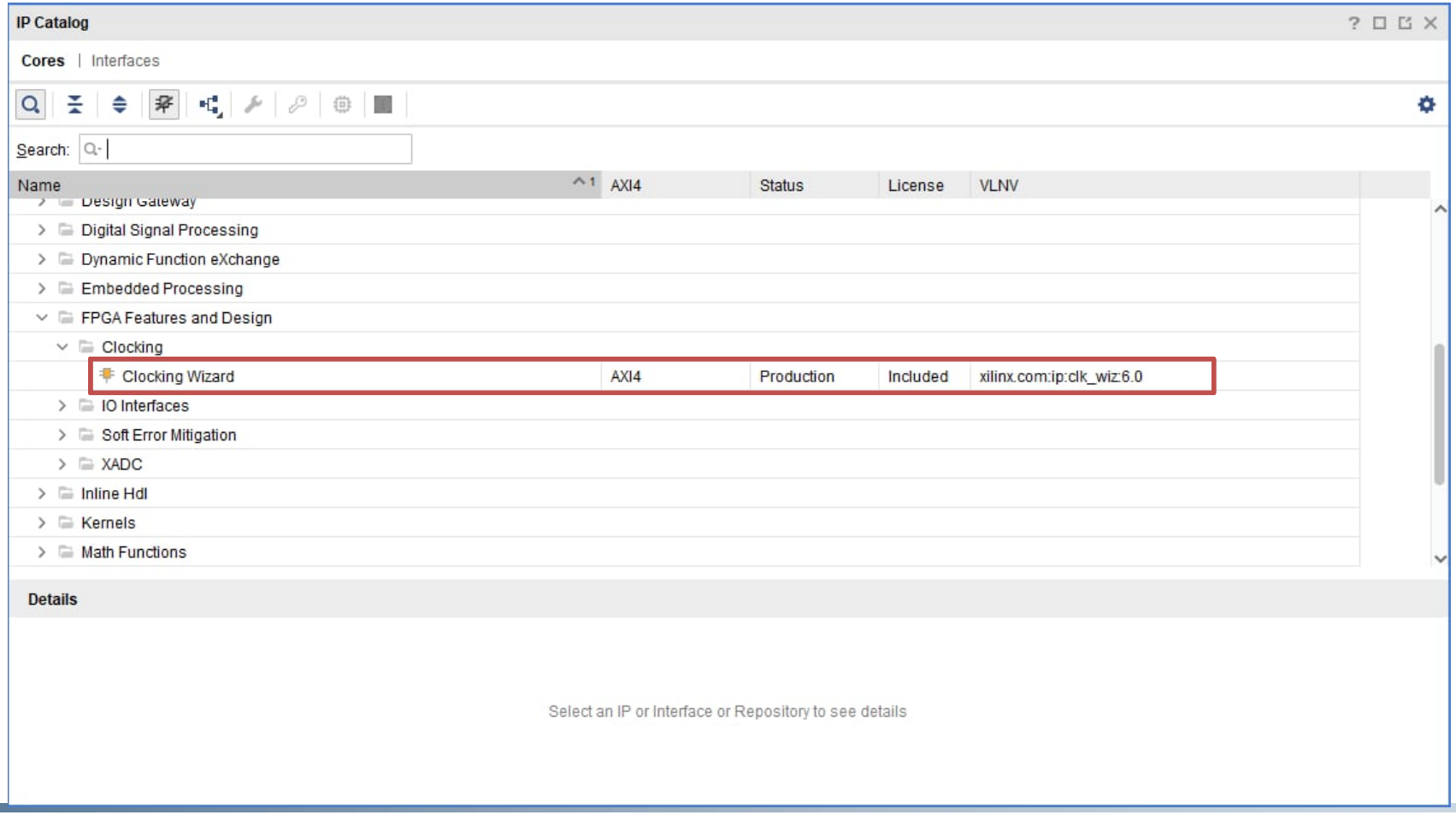


Clock

- Use the Clocking Wizard to generate the MMCM clocks.
  - The input clock is the onboard 100 MHz clock.
  - Generate a 200 MHz clock (clk_out1) and a 3.125 MHz clock (clk_out5).
  - It is recommended that all modules use the same clock, except for ddr2_interface.
  - Use the locked signal to initialize the system.
  - Do not implement your own clock divider using counters.

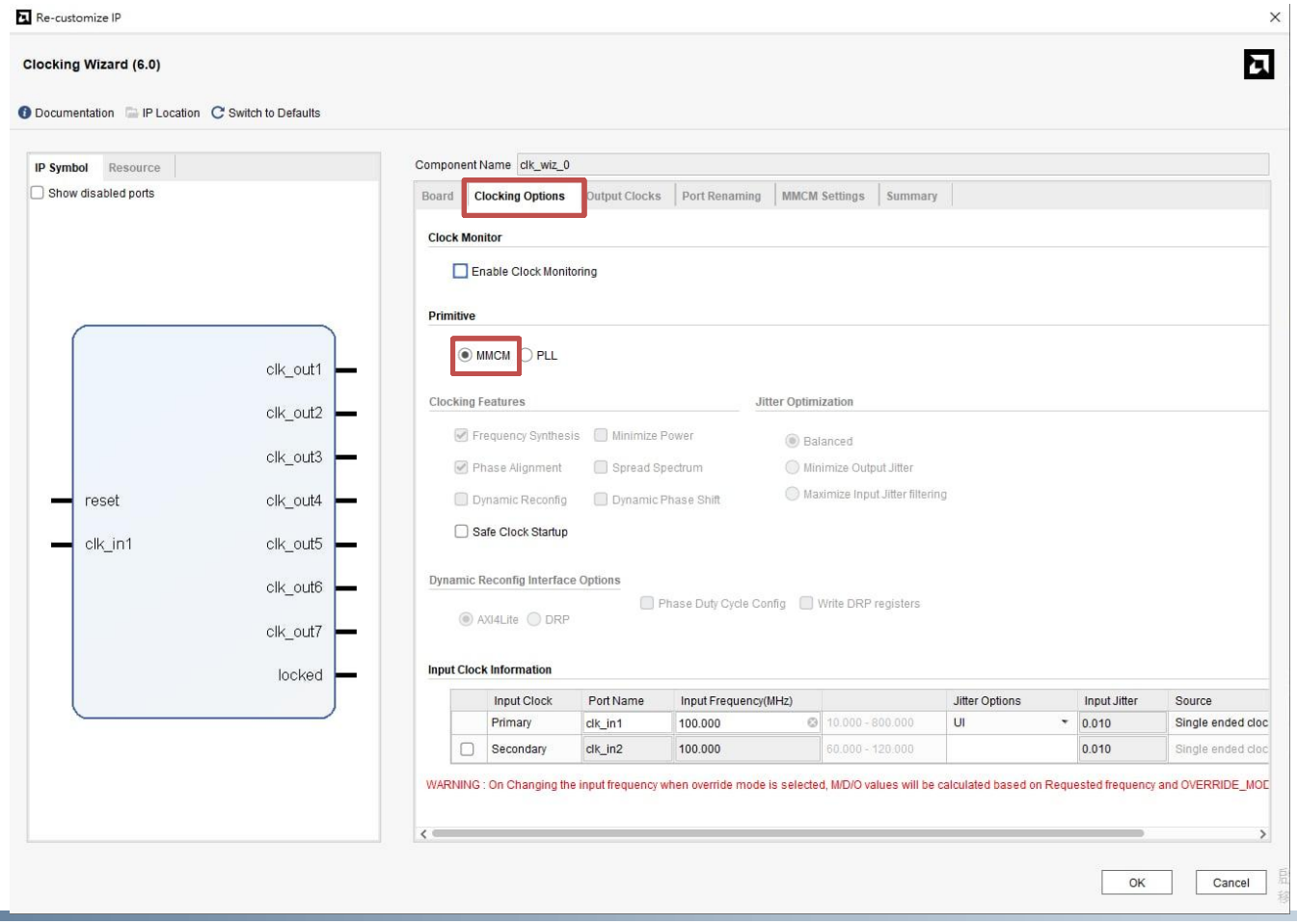
Clock

- IP Catalog => Clocking Wizard



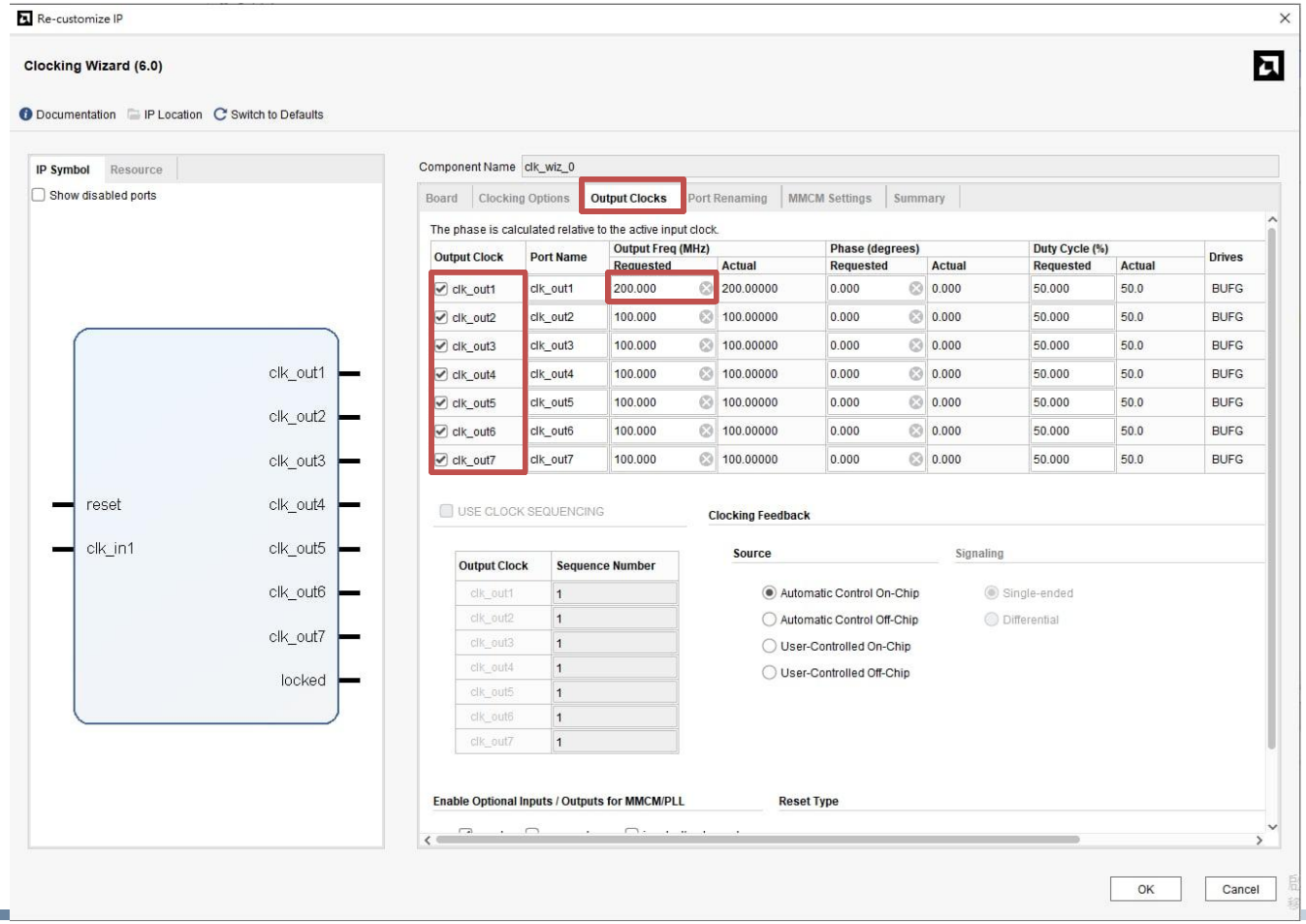
Clock

- Clocking Options => Check MMCM



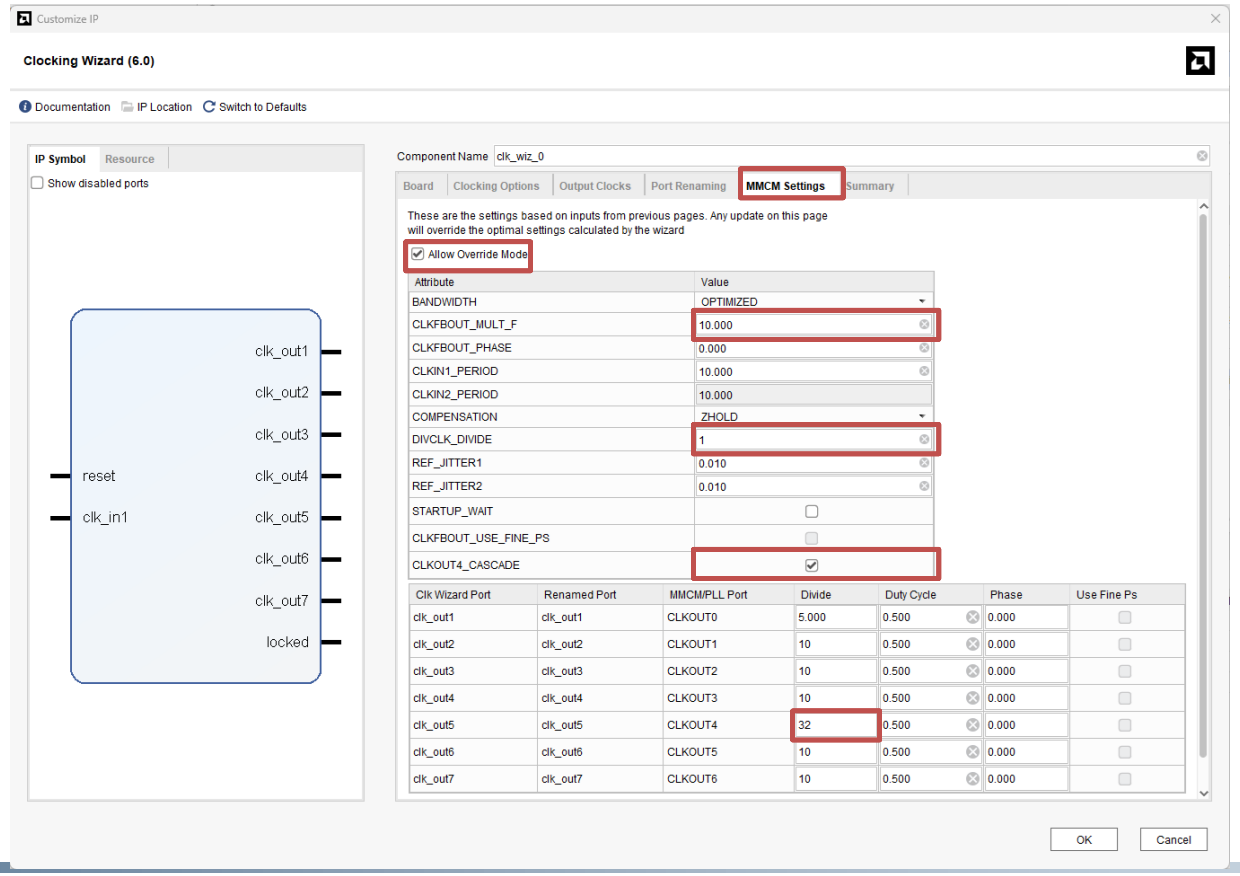
Clock

- Output Clocks => Check clk_out1~ clk_out7 => set clk_out1 to 200MHz



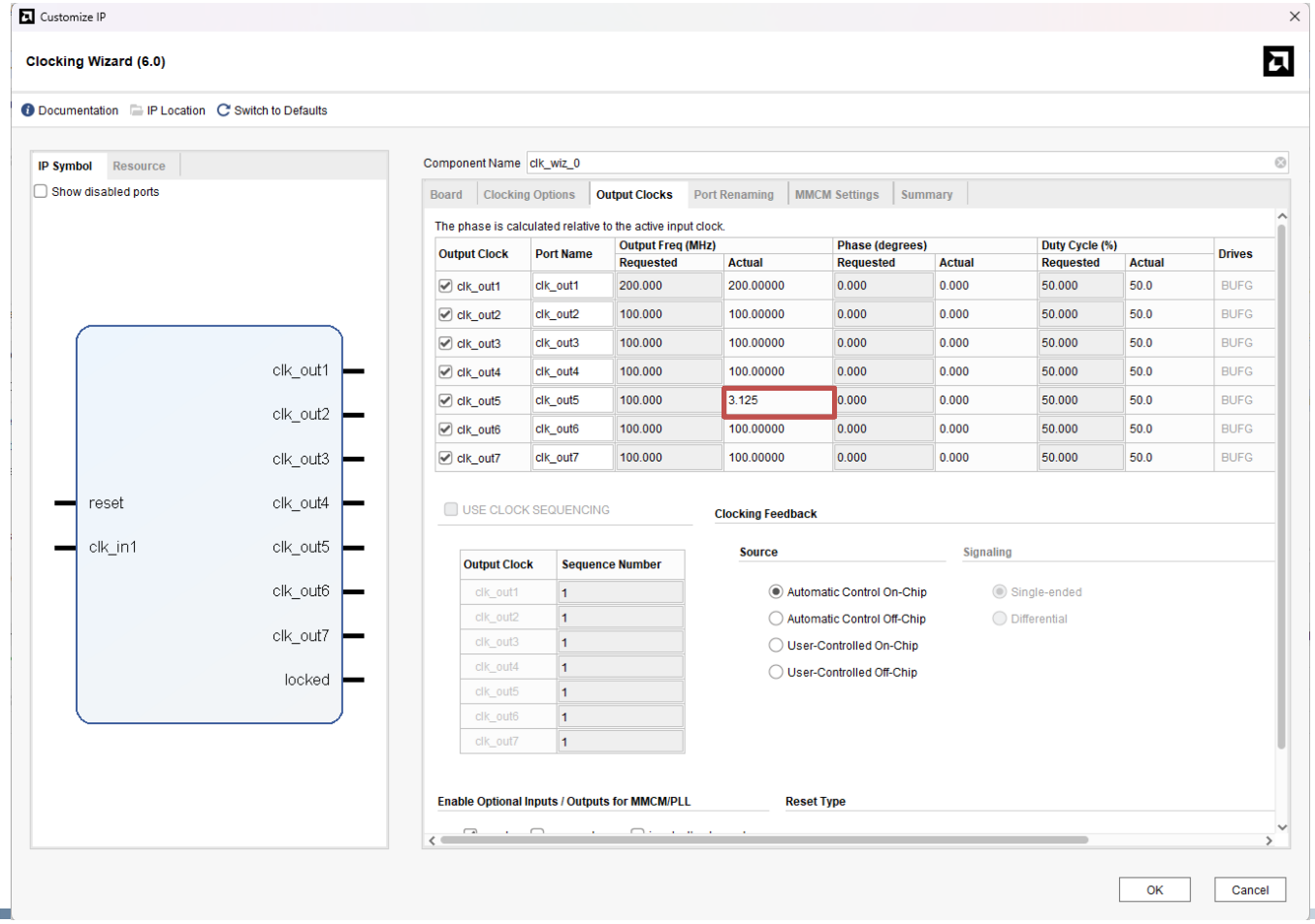
Clock

- MMCM Setting => Check Allow Overide Mode=> CLKBOUT_MULT_F = 10.000, DIVCLK_DIVIDE = 1=> Check CLKOUT4_CASCADE => clk_out5 Divide = 32



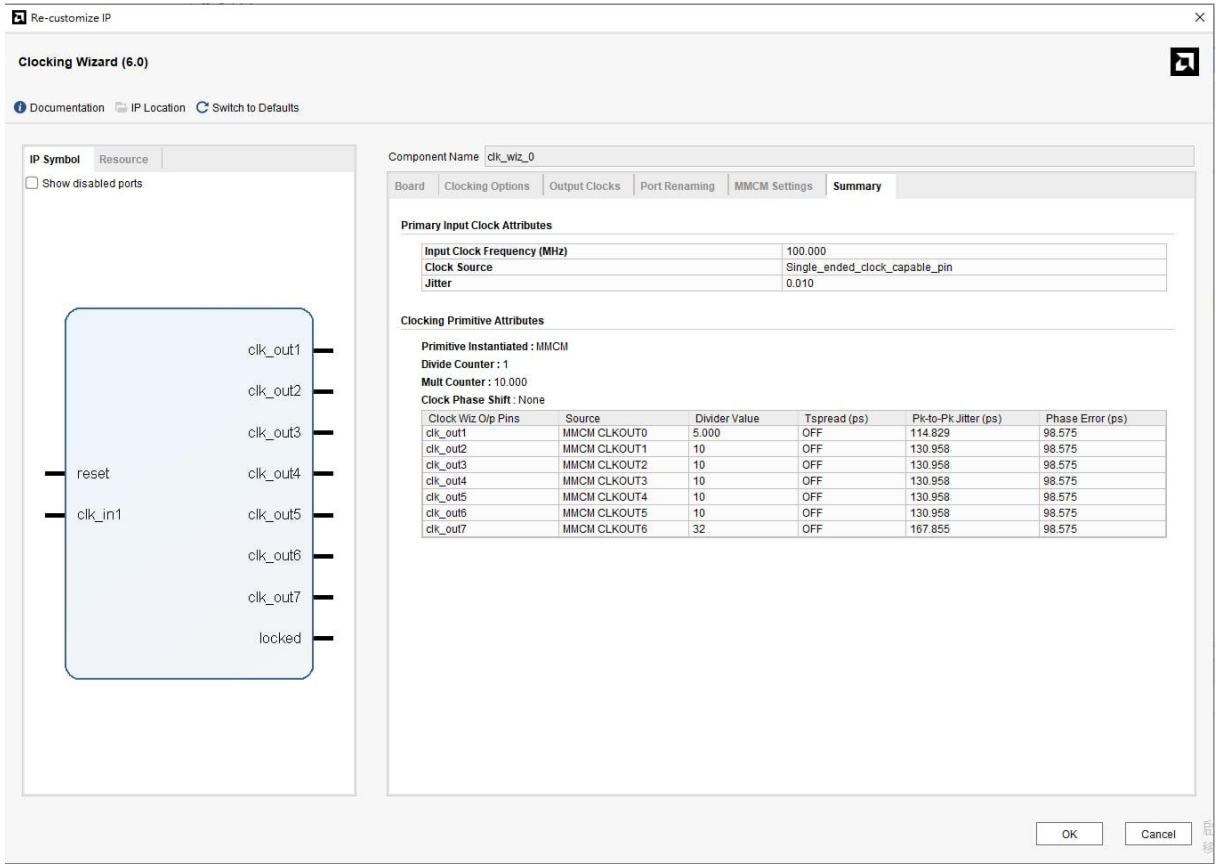
Clock

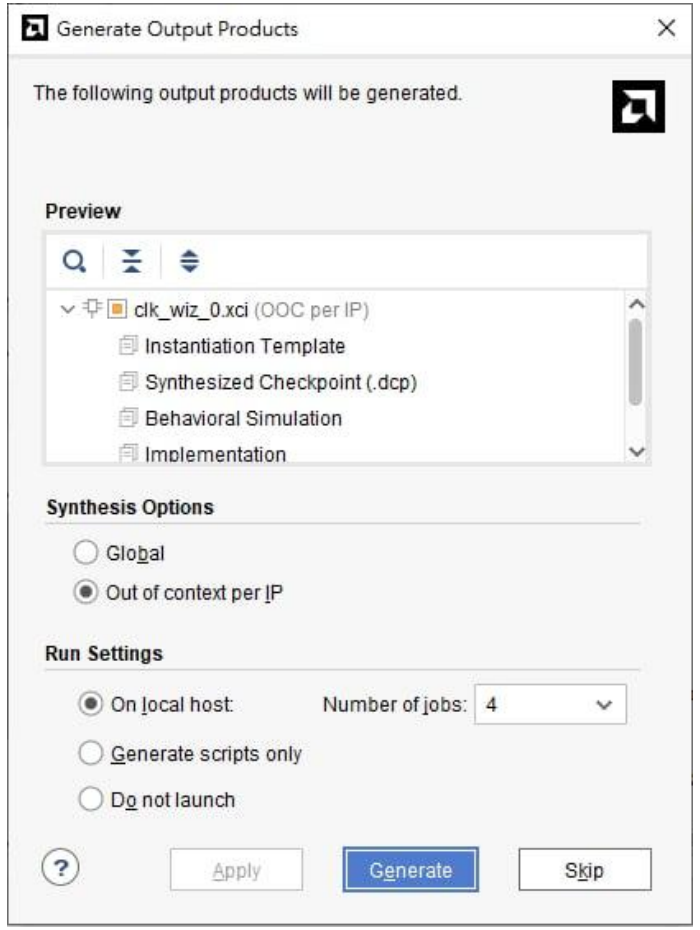
- Output Clocks => Make sure Actual Output Freq of clk_out5 = 3.125MHz



Clock

- OK => Generate







Outline

- Introduction
  - Lab requirements
- Audio CODEC
- Memory devices
- Implementation
  - System architecture
  - Work with WM8731
  - Digital signal processing (DSP)
  - Clock
- Code template
- Report regulations

Code Template

- Nexys-A7-100T-Master.xdc
  - Map top-level I/O to FPGA physical I/O
- NEXYS_A7.sv
  - Top module mapped to FPGA
- Top.sv
  - Handles the data processing for fast and slow playback.
  - Stores the received audio data in DDR2 memory.
  - You may modify the design as needed.
- ddr2_interface.sv
  - Receives read/write requests from Top and communicates with the DDR2 memory through the MIG interface.
- mig.ucf
  - Constraint file required for the MIG IP.
- Recommendations
  - Develop testbenches to verify each module individually.
  - Integrate the modules only after confirming that each one functions correctly.



Outline

- Introduction
  - Lab requirements
- Audio CODEC
- Memory devices
- Implementation
  - System architecture
  - Work with WM8731
  - Digital signal processing (DSP)
  - Clock
- Code template
- Report regulations



Report Regulations

- The report should include the following:
  - File Structure
  - System Architecture (with the Data Path clearly labeled)
  - Hardware Scheduling (FSM or Algorithm Workflow)
  - Screenshot of the Report Timing Summary
  - Screenshot of the Report Utilization
  - Problems encountered and their solutions, reflections and suggestions
- One report per team. Submit a PDF file.
- Filename format: teamXX_lab3_report.pdf
  - Ex: team01_lab3_report.pdf
- Submission deadline: 11:59 PM on the day of the demo.
  - Late submission policy: The score will be multiplied by 0.7 for every day late.



Submission Rules

- Submission File Structure

```
team01_lab3
|- team01_lab3_report.pdf
|- NEXYS_A7
   |- <All folders and files under project directory>
```

- Compress the teamXX_lab3 folder into a ZIP file and upload it to NTU Cool. One submission per team.
- The NEXYS_A7 project folder must contain all files required for the TA to reproduce the results by opening the project file (NEXYS_A7.xpr).
- Submission deadline: **11:59 PM on the day of the demo.**
- Failure to follow the required submission format may result in a grade deduction.



Grading Policy

- 15% of the Total Grade
  - Demo: 10%
  - Report & Code: 2%
  - Performance (Slice Utilization): 3%
    - Baseline (Slice < 1320): 1.5%
    - Ranking: 1.5%
- No plagiarism, any plagiarism will result in 0 points.
- Do not modify the constraint file. (Nexys-A7-100T-Master.xdc)



Questions?